

Laboratorio de Sistemas Móviles y Ubicuos

Grado en Ingeniería Telemática

Patrón Observer (LiveData)



Universidad
Rey Juan Carlos

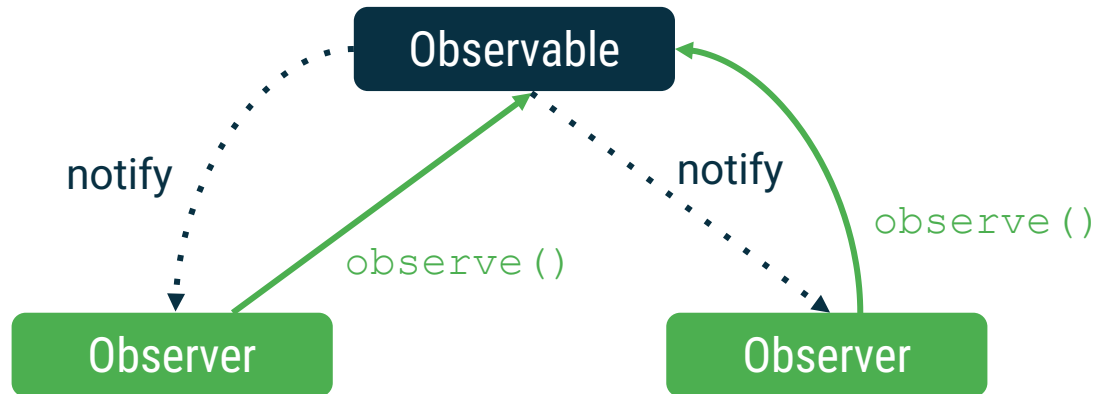
Observer

Patrón de diseño *Observer*

La vinculación de datos (DataBinding) ha reducido algunas de las asignaciones y llamadas a funciones en nuestras interfaces de usuario. Sin embargo, todavía tenemos que actualizar explícitamente las vistas con los nuevos valores del ViewModel cuando cambian. Hay una forma mejor.

El patrón de diseño *Observer* consiste en que un objeto (sujeto u observable) mantiene una lista de objetos dependientes (observadores) para notificar cuando se producen cambios de estado en el sujeto.

Patrón de diseño *Observer* (cont)



- El sujeto (*observable*) mantiene una lista de observadores a los que notificar los cambios de estado.
- Los observadores reciben los cambios de estado del sujeto y ejecutan el código apropiado.
- Los observadores pueden añadirse o eliminarse en cualquier momento.

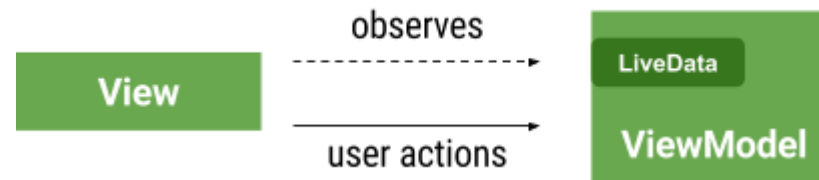
LiveData

LiveData

LiveData es una envoltura alrededor de cualquier tipo de datos (`LiveData<Int>` contiene un *Int*). De acuerdo con el patrón *Observer*, LiveData **es el sujeto observable**.

Los observadores serían controladores de interfaz de usuario, como Activity o Fragment, que esperan cambios en los datos subyacentes para actualizar la interfaz.

- Es un soporte de datos consciente del ciclo de vida que puede ser observado
- A menudo utilizado por ViewModels para contener campos de datos individuales



LiveData

LiveData no tiene métodos públicos para actualizar los datos almacenados. Se puede pensar que es "sólo lectura". Por otro lado, la clase *MutableLiveData* expone los métodos públicos *setValue(T)* y *postValue(T)*.

Habitualmente, *MutableLiveData* es utilizado como una variable privada en el *ViewModel*, que sólo expone un objeto *LiveData* inmutable a los observadores.

Así, el *ViewModel* es el responsable de implementar cualquier lógica y actualizar el *MutableLiveData*, mientras que el papel de la interfaz de usuario es sólo reaccionar a los cambios de valor de *LiveData*.

LiveData<T>	MutableLiveData<T>
● <code>getValue()</code>	● <code>getValue()</code> ● <code>postValue(value: T)</code> ● <code>setValue(value: T)</code>

LiveData en ViewModel

Se actualiza el código del *ViewModel* para utilizar *LiveData<T>* en lugar del tipo de dato *T* original.

Para evitar que ningún objeto externo u observable cambie los datos del *ViewModel*, se declara un *MutableLiveData* privado y un *LiveData* público que no pueden ser reasignados.

```
class LoginViewModel: ViewModel() {  
    private val _nombre = MutableLiveData<String>("Jorge")  
    val nombre: LiveData<String>  
        get() = _nombre  
  
    // La misma estructura pero para contraseña [...]  
  
    fun cambiarNombre(nuevo: String){  
        _nombre.value = nuevo  
    }  
}
```

LoginViewModel.kt

Observando cambios en *LiveData*

En la Actividad, creamos un observador para el dato en cuestión, implementando la lógica a ejecutar cuando se reciba la notificación de un cambio. Luego, se asocia al campo observable del ViewModel.

Las acciones del usuario que modifiquen los datos subyacentes deben lanzar llamadas a métodos del ViewModel (por ejemplo, el click en un botón).

```
override fun onCreate(savedInstanceState: Bundle?) {  
    super.onCreate(savedInstanceState)  
    binding = DataBindingUtil setContentView(this, R.layout.activity_main)  
    binding.viewModel = viewModel  
    binding.button.setOnClickListener {  
        viewModel.cambiarNombre("Laura")  
    }  
  
    val nombre_observer = Observer<String> { newValue ->  
        binding.usuario.text = newValue.toString()  
    }  
    viewModel.nombre.observe(this, nombre_observer)  
}
```

MainActivity.kt

El primer parámetro de *observe* es el *lifecycleOwner*, que indica a qué ciclo de vida se asocia el observador (una actividad o fragmento)

Ejercicio 1

Modifica la aplicación del marcador de un partido de baloncesto, desarrollada en la sesión anterior, para que al pulsar los botones se modifiquen los puntos del equipo correspondiente:

- Adapta el ViewModel para que los datos estén contenidos en LiveData
- Añade observadores en la actividad principal para reaccionar ante los cambios en los datos del ViewModel



Asociación directa Layout-LiveData

Utilizando *DataBinding* podemos ahorrarnos la declaración explícita de observadores en la actividad.

De esta forma, podemos conectar directamente una vista del layout (.xml) con el valor del sujeto observable (LiveData), consiguiendo que la vista se actualice cuando el dato observado cambie.

Para ello, hay que introducir cambios tanto en el diseño en .xml como en la lógica de nuestra actividad.

Conectar el XML con datos observables

Si asignamos a una vista el valor del atributo LiveData del ViewModel (accesible desde el XML) se elimina la necesidad de declarar un observador en la Actividad.

```
<layout>
  <data>
    <variable name="viewModel" type="urjc.lsmu.viewmodelsimple.LoginViewModel" />
  </data>

  <ConstraintLayout ...>

    <TextView
      android:id="@+id/usuario"
      android:layout_width="wrap_content"
      android:layout_height="wrap_content"
      android:text="@{viewModel.nombre.toString()}" />

    ...
  </ConstraintLayout>
</layout>
```

MainActivity.kt

Definir el ciclo de vida de los observadores

Después de asignar el objeto *viewModel* al *DataBinding* para hacerlo accesible desde nuestro layout (.xml), hay que especificar el *lifecycleOwner* (la actividad o fragmento) al que va asociado, tal y como se hace al declarar los *observers* de forma explícita.

```
override fun onCreate(savedInstanceState: Bundle?) {  
    super.onCreate(savedInstanceState)  
    binding = DataBindingUtil.setContentViews(this, R.layout.activity_main)  
    binding.viewModel = viewModel  
    binding.lifecycleOwner = this  
  
    binding.button.setOnClickListener {  
        viewModel.cambiarNombre("Laura")  
    }  
}
```

MainActivity.kt

El funcionamiento de esta actividad es equivalente al de la anterior. Ahora, los observadores se generan implícitamente en el *DataBinding*.

Ejercicio 2

Siguiendo con el ejercicio anterior, modifica la aplicación para hacer un enlace directo entre la vista (xml) y los datos del ViewModel sin necesidad de declarar observadores en la actividad principal.



Ejercicio 3

Crea una aplicación para gestionar una lista de tareas. Sus requisitos son:

1. La app debe tener dos actividades:
 - Una actividad principal que muestra la lista de tareas y tiene un FAB que permite lanzar la actividad secundaria.
 - Una actividad secundaria para añadir tareas nuevas a la lista o editar las ya existentes. Ambas operaciones deben poder ser canceladas.
2. Cada tarea debe contar con un título, una descripción, y un estado (completada/por hacer):
 - En la lista de la actividad principal cada item mostrará únicamente una checkbox indicando si está completada, y el título de la tarea.
 - Si se hace click en la checkbox, se debe cambiar su estado. Si se hace click en el título, se debe lanzar la actividad para editar la tarea.
3. La app debe implementarse utilizando ViewModel, DataBinding y LiveData.

Referencias

- *Curso Android Development with Kotlin*
- <https://developer.android.com/>